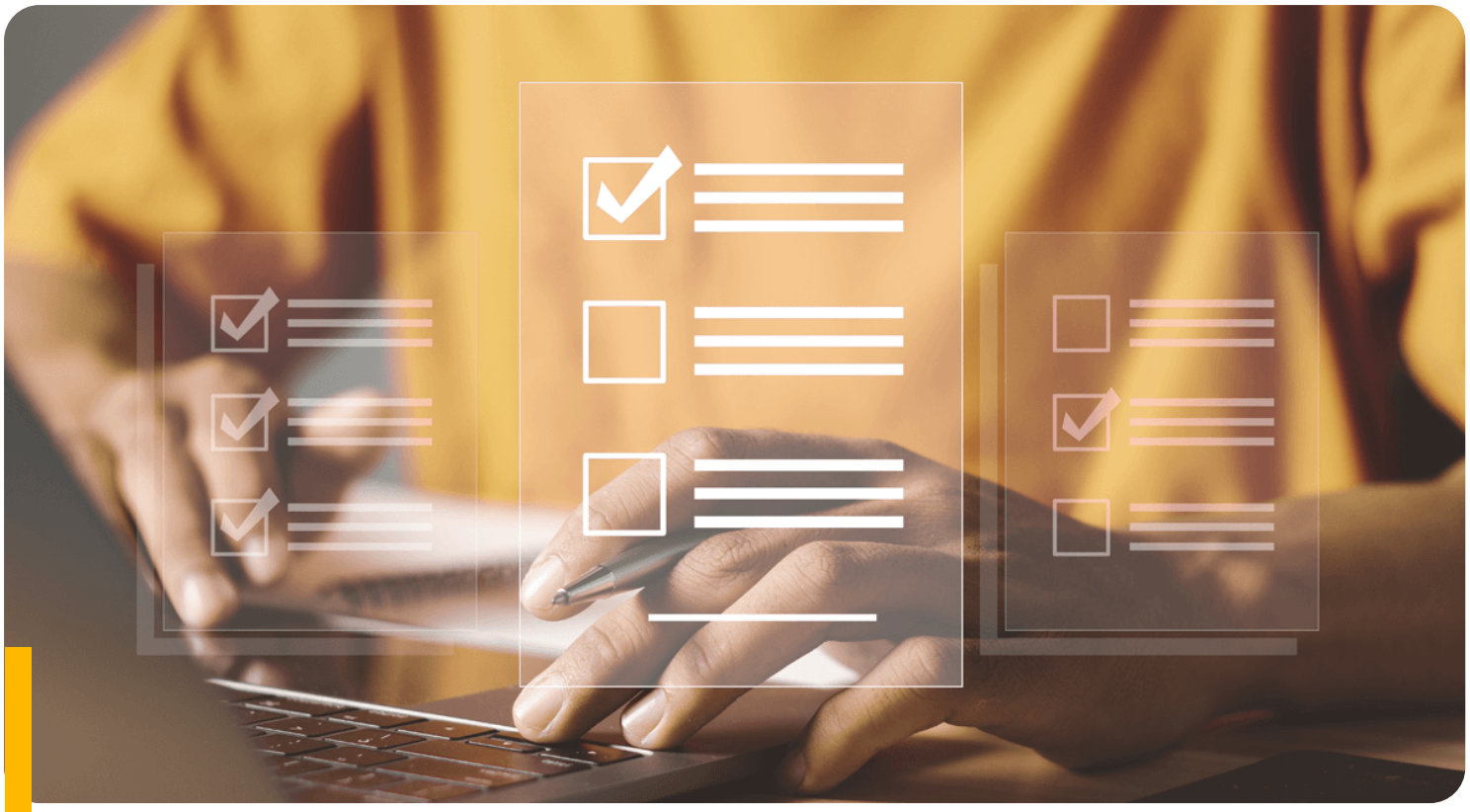
DuocUC | ONLINE



(PBY2203)

Nombre estudiante: Raúl Zúñiga	
Asignatura: Desarrollo Backend III	**Carrera: Desarrollo de Aplicaciones**
Profesor: Gabriel Grobier	**Fecha:14-09-2026**

Descripción de la actividad

En esta quinta semana realizarás una actividad sumativa individual titulada "Implementando el patrón arquitectónico Backend for Frontend (BFF)" donde deberás implementar un sistema utilizando el patrón Backend for Frontend (BFF) para optimizar la comunicación entre distintos frontends de la aplicación del Banco XYZ. El objetivo será crear un backend personalizado para cada tipo de cliente (web, móvil y cajeros) que permita manejar eficientemente las solicitudes y mejorar la experiencia del usuario en cada plataforma. El proyecto debe cumplir con los requisitos para garantizar la integridad y consistencia de los datos, aplicando el patrón Backend for Frontend.

Instrucciones específicas

En función de la actividad formativa de la semana 4 llamada Analizando el patrón arquitectónico con Backend for Frontend (BFF), deberás avanzar en el proyecto en su continuidad, realizando:

1. Analizar la estrategia de implementación de BFF
Debes determinar que estrategia de implementación es la que más conviene al proyecto solicitado.

2. Implementar BFF para cada cliente
Cada cliente deberá tener su propio Backend.

3. Configuraciones de los BFF
Tu implementación deberá considerar HTTPS, Autenticación y Autorización.

Requerimientos del Sistema

Basándote en los datos de https://github.com/KariVillagran/bank_legacy_data deberás crear las APIs correspondientes para acceder a los datos.

Configura un Backend para cada uno de los tres clientes:

- **Implementar BFF Web**: Optimizado para navegadores, proporciona datos completos y soporte para interfaces complejas.
- **Implementar BFF Móvil**: Respuestas ligeras, datos esenciales para reducir consumo de ancho de banda y mejorar la velocidad.
- **Implementar BFF Cajeros Automáticos**: Interfaz segura y eficiente para operaciones críticas como retiros y consultas de saldo.
- Gestionar autenticación y autorización específicas para cada canal.

Entrega

- **Código Fuente:** Sube el código del proyecto a un repositorio de GitHub con su propuesta técnica
- **Documentación** (README.md): Describe el objetivo del proyecto, la estructura del código y las instrucciones para ejecutar el proyecto.
- **Evidencia de Ejecución:** Proporciona capturas de pantalla o salidas de consola mostrando la ejecución de cada API y los resultados generados.

Desarrollo

Para dar cumplimiento a esta semana, se desarrolló 3 BFF los cuales se encuentran separados al interior del paquete bff, en su interior se encuentran las clases controladoras llamadas AtmBffController.java, MobileBffController.java, WebBffController.java.

AtmBffController.java

```java
package com.backen3.Duoc.bff;

import com.backen3.Duoc.dto.AtmBalanceResponse;
import com.backen3.Duoc.dto.AtmWithdrawalRequest;
import com.backen3.Duoc.dto.AtmWithdrawalResponse;
import com.backen3.Duoc.service.BffBankingService;
import org.springframework.http.HttpStatus;
import org.springframework.web.bind.annotation.ExceptionHandler;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.RequestBody;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.ResponseStatus;
import org.springframework.web.bind.annotation.RestController;

@RestController
@RequestMapping("/api/bff/atm")
public class AtmBffController {

    private final BffBankingService bffBankingService;

    public AtmBffController(BffBankingService bffBankingService) {
        this.bffBankingService = bffBankingService;
    }

    @GetMapping("/cuentas/{cuentaId}/saldo")
    public AtmBalanceResponse saldo(@PathVariable Long cuentaId) {
        return bffBankingService.getAtmBalance(cuentaId);
    }

    @PostMapping("/cuentas/{cuentaId}/retiro")
    public AtmWithdrawalResponse retiro(@PathVariable Long cuentaId, @RequestBody AtmWithdrawalRequest request) {
        return bffBankingService.withdrawFromAtm(cuentaId, request.monto());
    }

    @ExceptionHandler(IllegalArgumentException.class)
    @ResponseStatus(HttpStatus.BAD_REQUEST)
    public ErrorResponse handleIllegalArgument(IllegalArgumentException exception) {
        return new ErrorResponse(exception.getMessage());
    }

    public record ErrorResponse(String error) {
    }
}
```

MobileBffController.java

```java
package com.backen3.Duoc.bff;

import com.backen3.Duoc.dto.MobileAccountResponse;
import com.backen3.Duoc.dto.MobileMovementResponse;
import com.backen3.Duoc.service.BffBankingService;
import java.util.List;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;

@RestController
@RequestMapping("/api/bff/mobile")
public class MobileBffController {

    private final BffBankingService bffBankingService;

    public MobileBffController(BffBankingService bffBankingService) {
        this.bffBankingService = bffBankingService;
    }

    @GetMapping("/cuentas")
    public List<MobileAccountResponse> cuentas() {
        return bffBankingService.getMobileAccounts();
    }

    @GetMapping("/cuentas/{cuentaId}/movimientos")
    public List<MobileMovementResponse> movimientos(@PathVariable Long cuentaId) {
        return bffBankingService.getMobileLatestMovements(cuentaId);
    }
}
```

WebBffController.java

```java
package com.backen3.Duoc.bff;

import com.backen3.Duoc.dto.WebAccountDetailResponse;
import com.backen3.Duoc.dto.WebAnnualReportResponse;
import com.backen3.Duoc.dto.WebDailySummaryResponse;
import com.backen3.Duoc.service.BffBankingService;
import java.util.List;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;

@RestController
@RequestMapping("/api/bff/web")
public class WebBffController {

    private final BffBankingService bffBankingService;

    public WebBffController(BffBankingService bffBankingService) {
        this.bffBankingService = bffBankingService;
    }

    @GetMapping("/cuentas")
    public List<WebAccountDetailResponse> cuentas() {
        return bffBankingService.getWebAccounts();
    }

    @GetMapping("/resumen-transacciones")
    public List<WebDailySummaryResponse> resumenTransacciones() {
        return bffBankingService.getWebDailySummaries();
    }

    @GetMapping("/informe-anual")
    public List<WebAnnualReportResponse> informeAnual() {
        return bffBankingService.getWebAnnualReports();
    }
}

```

Posteriormente se creó en el paquete DTO, distintas clases de dtos para cada canal, con el objetivo de demostrar unos payload completo vs liviano y vs críticos

AtmBalanceResponse.java

src > main > java > com > backen3 > Duoc > dto > J

```java
package com.backen3.Duoc.dto;

import java.math.BigDecimal;

public record AtmBalanceResponse(
        Long cuentaId,
        BigDecimal saldoDisponible,
        String estado,
        String canal) {
}

```

AtmWhitdrawalRequest.java

```java
package com.backen3.Duoc.dto;

import java.math.BigDecimal;

public record AtmWithdrawalRequest(BigDecimal monto) {
}

```

AtmWhitdrawalResponse.java

```java
package com.backen3.Duoc.dto;

import java.math.BigDecimal;

public record AtmWithdrawalResponse(
        Long cuentaId,
        BigDecimal montoRetirado,
        BigDecimal saldoDisponible,
        String autorizacion,
        String mensaje) {
}

```

MobileAccountResponse.java

```java
package com.backen3.Duoc.dto;

import java.math.BigDecimal;

public record MobileAccountResponse(
        Long cuentaId,
        String nombre,
        BigDecimal saldo,
        String tipo) {
}
```

MobileMovementResponse.java

```java
package com.backen3.Duoc.dto;

import java.math.BigDecimal;
import java.time.LocalDate;

public record MobileMovementResponse(
        LocalDate fecha,
        BigDecimal monto,
        String tipo) {
}

```

WebAccountDetailResponse.java

```java
package com.backen3.Duoc.dto;

import java.math.BigDecimal;

public record WebAccountDetailResponse(
        Long cuentaId,
        String nombre,
        Integer edad,
        String tipo,
        BigDecimal saldoInicial,
        BigDecimal tasaInteres,
        BigDecimal interesCalculado,
        BigDecimal saldoFinal) {
}

```

WebAnnualReportResponse.java

```java
package com.backen3.Duoc.dto;

import java.math.BigDecimal;

public record WebAnnualReportResponse(
        Long cuentaId,
        Integer anio,
        BigDecimal totalDepositos,
        BigDecimal totalRetirosCompras,
        BigDecimal saldoNeto,
        Integer cantidadMovimientos) {
}
```

WebDailySummaryResponse.java

```java
package com.backen3.Duoc.dto;

import java.math.BigDecimal;
import java.time.LocalDate;

public record WebDailySummaryResponse(
        LocalDate fecha,
        BigDecimal totalDebitos,
        BigDecimal totalCreditos,
        Integer totalMovimientos,
        Integer cantidadAnomalias) {
}

```

Para la autenticación por canal, se utilizó el header X-Channel-Token para ello utilicé los tokens simples por canal BFF, con el fin de demostrar que cada backend tiene una autentificación propia según el tipo de cliente, ya sea web, móvil o de un cajero automático. Ojo que esto no es una seguridad como lo es JWT o la utilización de un OAuth, pero para esta actividad nos va a servir muy bien para evidenciar la gestión de la autentificación y autorización específica de cada canal.
En el application.propperties queda así:

```
25
26    # Tokens simples por canal BFF para demostracion.
27    app.security.tokens.web=${BFF_WEB_TOKEN:web-token-xyz}
28    app.security.tokens.mobile=${BFF_MOBILE_TOKEN:mobile-token-xyz}
29    app.security.tokens.atm=${BFF_ATM_TOKEN:atm-token-xyz}
30
```

Esto significa para BFF web, el token esperado es web-token-xyz, para BFF móvil, el token esperado es mobile-token-xyz y para el cajero es atm-token-xyz.
Ahora en el paquete config, la clase ChannelAuthInterceptor.java, con esto spring inyectará esos valores con @Value:

```
@Component
public class ChannelAuthInterceptor implements HandlerInterceptor {

    private final Map<String, String> tokensByChannel;

    public ChannelAuthInterceptor(
            @Value("${app.security.tokens.web}") String webToken,
            @Value("${app.security.tokens.mobile}") String mobileToken,
            @Value("${app.security.tokens.atm}") String atmToken) {
        this.tokensByChannel = Map.of(
                "web", webToken,
                "mobile", mobileToken,
                "atm", atmToken);
    }
```

Spring Inyecta esos valores en @Value y después los guarda en un mapa

Cuando llega la petición, el interceptor revisa la URL. Por ejemplo:
/api/bff/web/cuentas
Aquí detecta que pertenece al canal web. Pero si la URL empieza con /api/bff/mobile/, va a detectar que pertenece a mobile; si empieza con /api/bff/atm/, va a detectar que pertenece a atm.
Luego va a comparar los token esperado contra el header que mandamos en el curl:
String providedToken = request.getHeader("X-Channel-Token");
Por esta razón cuando los llamemos en consola debemos hacerlo con **curl -H "X-Channel-Token: web-token-xyz"** http://localhost:8080/api/bff/web/cuentas. Por lo tanto, si el token coincide dejará pasar la solicitud, sin embargo, si no coincide o falta nos devolverá un JSON

```
{"error":"No autorizado","detalle":"Token de canal invalido o ausente"}
```

Finalmente en el paquete config la clase WebConfig.java tiene un interceptor que aplicará a todas las rutas del BFF

```java
package com.backen3.Duoc.config;

import org.springframework.context.annotation.Configuration;
import org.springframework.web.servlet.config.annotation.InterceptorRegistry;
import org.springframework.web.servlet.config.annotation.WebMvcConfigurer;

@Configuration
public class WebConfig implements WebMvcConfigurer {

    private final ChannelAuthInterceptor channelAuthInterceptor;

    public WebConfig(ChannelAuthInterceptor channelAuthInterceptor) {
        this.channelAuthInterceptor = channelAuthInterceptor;
    }

    @Override
    public void addInterceptors(InterceptorRegistry registry) {
        registry.addInterceptor(channelAuthInterceptor)
                .addPathPatterns("/api/bff/**");
    }
}

```

Ahora para hacer la demostración del funcionamiento, debo ejecutar mi aplicación y después en la terminal escribo lo siguiente:

curl -H "X-Channel-Token: web-token-xyz" http://localhost:8080/api/bff/web/cuentas

raulzunigagonzalez — -zsh — 80×24

```
Last login: Mon Aug 31 20:25:29 on ttys002
raulzunigagonzalez@MacBook-Air-de-Raul-2 ~ % curl -H "X-Channel-Token: web-token
-xyz" http://localhost:8080/api/bff/web/cuentas
[{"cuentaId":101,"nombre":"John Doe","edad":30,"tipo":"ahorro","saldoInicial":50
00.00,"tasaInteres":0.0150,"interesCalculado":75.00,"saldoFinal":5075.00},{"cuen
taId":102,"nombre":"Jane Smith","edad":25,"tipo":"prestamo","saldoInicial":8000.
00,"tasaInteres":0.0250,"interesCalculado":200.00,"saldoFinal":8200.00},{"cuenta
Id":103,"nombre":"Bob Johnson","edad":30,"tipo":"prestamo","saldoInicial":12000.
00,"tasaInteres":0.0250,"interesCalculado":300.00,"saldoFinal":12300.00},{"cuent
aId":104,"nombre":"Alice Brown","edad":45,"tipo":"ahorro","saldoInicial":0.00,"t
asaInteres":0.0150,"interesCalculado":0.00,"saldoFinal":0.00},{"cuentaId":105,"n
ombre":"Charlie Green","edad":35,"tipo":"hipoteca","saldoInicial":7000.00,"tasaI
nteres":0.0200,"interesCalculado":140.00,"saldoFinal":7140.00},{"cuentaId":106,"
nombre":"John Doe","edad":30,"tipo":"ahorro","saldoInicial":5000.00,"tasaInteres
":0.0150,"interesCalculado":75.00,"saldoFinal":5075.00},{"cuentaId":107,"nombre"
:"Diana Prince","edad":40,"tipo":"prestamo","saldoInicial":15000.00,"tasaInteres
":0.0250,"interesCalculado":375.00,"saldoFinal":15375.00},{"cuentaId":108,"nombr
e":"Steve Rogers","edad":80,"tipo":"ahorro","saldoInicial":10000.00,"tasaInteres
":0.0150,"interesCalculado":150.00,"saldoFraulzunigagonzalez@MacBook-Air-de-Raul
raulzunigagonzalez@MacBook-Air-de-Raul-2 ~ %
```

curl -H "X-Channel-Token: mobile-token-xyz" http://localhost:8080/api/bff/mobile/cuentas

```
raulzunigagonzalez@MacBook-Air-de-Raul-2 ~ % curl -H "X-Channel-Token: mobile-to
ken-xyz" http://localhost:8080/api/bff/mobile/cuentas
[{"cuentaId":101,"nombre":"John Doe","saldo":5075.00,"tipo":"ahorro"},{"cuentaId
":102,"nombre":"Jane Smith","saldo":8200.00,"tipo":"prestamo"},{"cuentaId":103,"
nombre":"Bob Johnson","saldo":12300.00,"tipo":"prestamo"},{"cuentaId":104,"nombr
e":"Alice Brown","saldo":0.00,"tipo":"ahorro"},{"cuentaId":105,"nombre":"Charlie
 Green","saldo":7140.00,"tipo":"hipoteca"},{"cuentaId":106,"nombre":"John Doe","
saldo":5075.00,"tipo":"ahorro"},{"cuentaId":107,"nombre":"Diana Prince","saldo":
15375.00,"tipo":"prestamo"},{"cuentaId":108,"nombre":"Steve Rogers","saldo":1015
0.00,"tipo":"ahorro"}]%
raulzunigagonzalez@MacBook-Air-de-Raul-2 ~ %
```

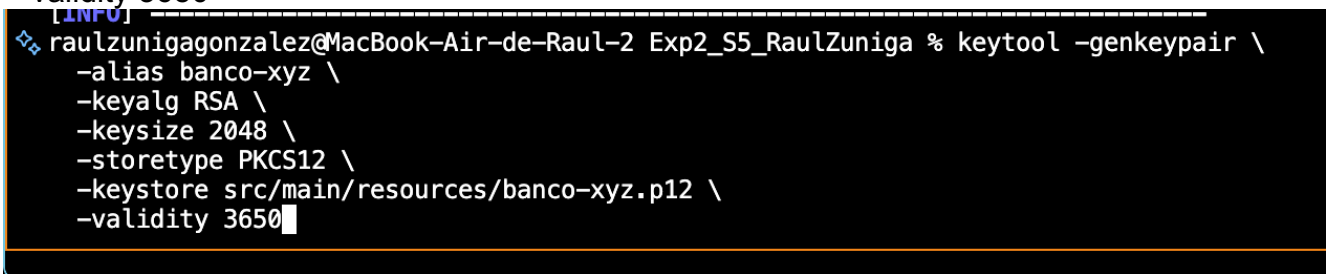
curl -H "X-Channel-Token: atm-token-xyz" http://localhost:8080/api/bff/atm/cuentas/101/saldo

raulzunigagonzalez — -zsh — 80×24

```
Last login: Mon Sep  7 13:08:30 on ttys002
raulzunigagonzalez@MacBook-Air-de-Raul-2 ~ % curl -H "X-Channel-Token: atm-token
-xyz" http://localhost:8080/api/bff/atm/cuentas/101/saldo
{"cuentaId":101,"saldoDisponible":5075.00,"estado":"OPERATIVA","canal":"ATM"}%
raulzunigagonzalez@MacBook-Air-de-Raul-2 ~ %
```

Ahora bien. Para esta semana se solicitó la generación de un certificado para las pruebas, razón por la cual desde la terminal de nuestro proyecto, debemos escribir el siguiente script:

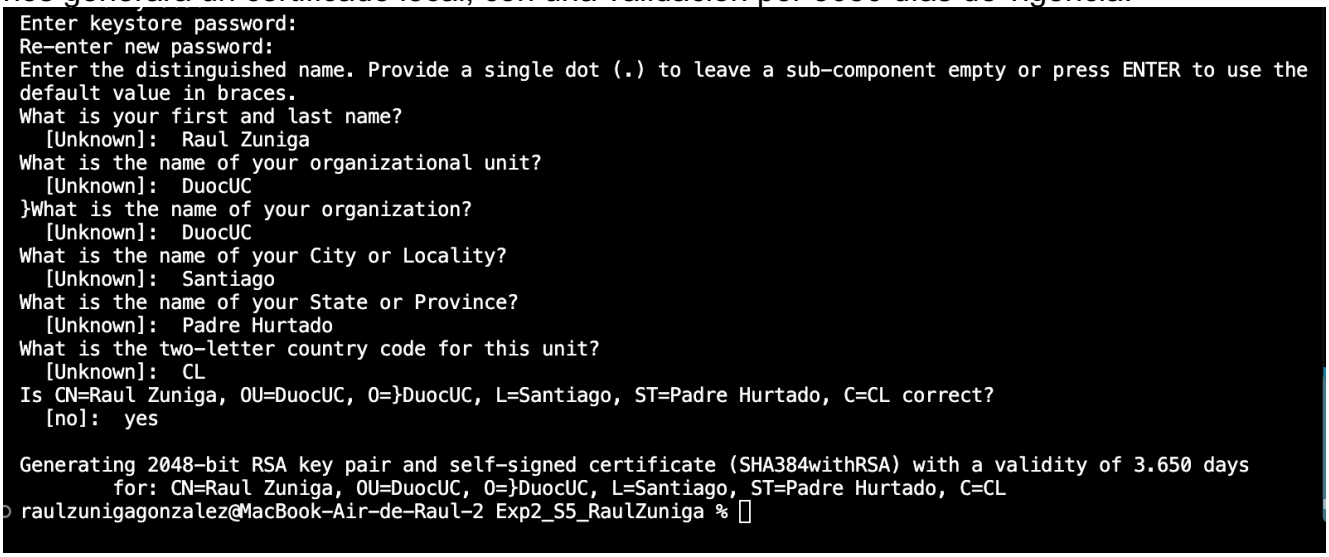
```
keytool -genkeypair \
  -alias banco-xyz \
  -keyalg RSA \
  -keysize 2048 \
  -storetype PKCS12 \
  -keystore src/main/resources/banco-xyz.p12 \
  -validity 3650
```



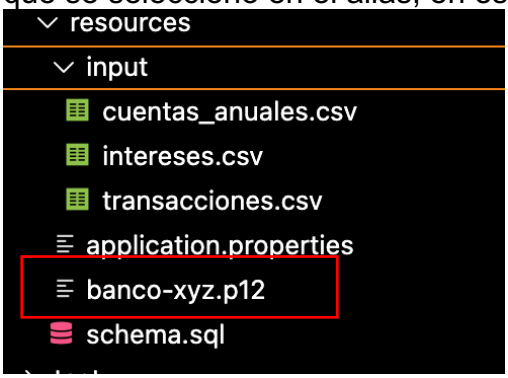
Una vez que el script se ejecute me pedirá algunos datos los cuales debemos ir contestando. En mi casao realicé lo siguiente:

1- Password: Backend3
2- Nombre: Raul Zuniga
3- Nombre de la Unidad de Organización: DuocUC
4- Nombre de mi organización: DuocUC
5- Nombre de la ciudad o localidad: Santiago
6- Nombre de la provincia: Padre Hurtado
7- Las dos letras del código del país: CL

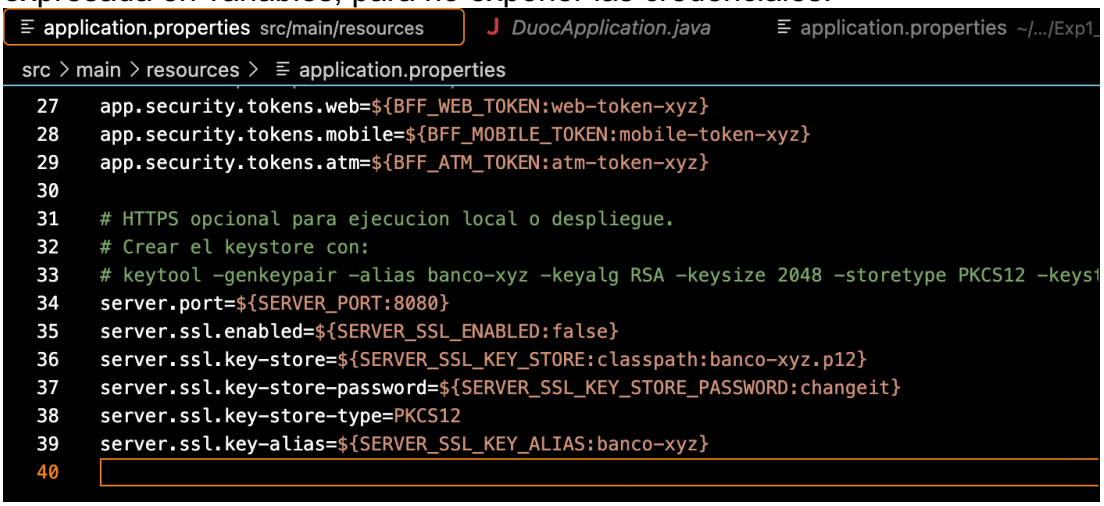
Después nos muestra todas las opciones que respondimos y finalizamos con un yes. Esto nos generará un certificado local, con una validación por 3650 días de vigencia.



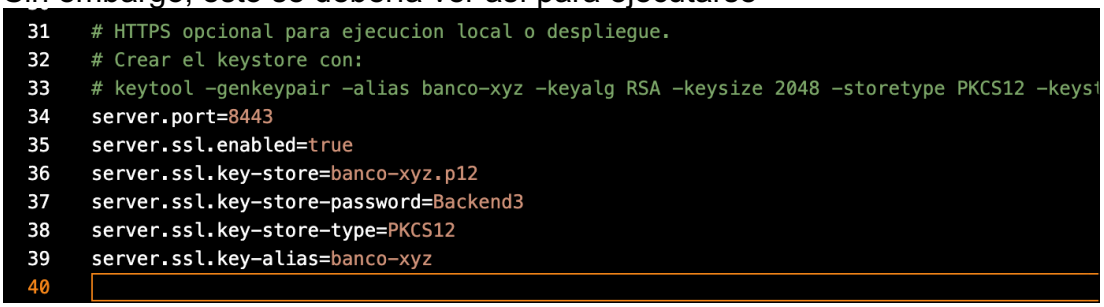
Ahora en nuestra carpeta resources, se habrá creado nuestro certificado .p12, con el nombre que se seleccionó en el alias, en este caso le puse banco-xyz.p12



Ahora vamos a ejecutar Spring Boot con HTTPS. ¿Cómo lo vamos a hacer? Pues en nuestro archivo application.propperties, vamos a encontrarnos con los parámetros necesarios para poder ejecutar el certificado. Sin embargo, cuando lo vea no encontrará la información expresada en variables, para no exponer las credenciales.



Sin embargo, esto se debería ver así para ejecutarse



Con los parámetros ya designados, procedemos a ejecutar la aplicación, escribiendo el siguiente script ./mvnw spring-boot:run -Dspring-boot.run.arguments="--fechaProceso=2024-01"

```
raulzunigagonzalez@MacBook-Air-de-Raul-2 Exp2_S5_RaulZuniga % ./mvnw spring-boot:run -Dspring-boot.run.arguments
="--fechaProceso=2024-01"
[INFO] Scanning for projects...
[INFO]
[INFO] ------------------------< com.backen3:Duoc >------------------------
[INFO] Building  0.0.1-SNAPSHOT
[INFO]   from pom.xml
[INFO] --------------------------------[ jar ]---------------------------------
[INFO]
[INFO] >>> spring-boot:3.5.7:run (default-cli) > test-compile @ Duoc >>>
[INFO]
[INFO] --- resources:3.3.1:resources (default-resources) @ Duoc ---
[INFO] Copying 1 resource from src/main/resources to target/classes
[INFO] Copying 5 resources from src/main/resources to target/classes
[INFO]
[INFO] --- compiler:3.14.1:compile (default-compile) @ Duoc ---
[INFO] Nothing to compile - all classes are up to date.
[INFO]
[INFO] --- resources:3.3.1:testResources (default-testResources) @ Duoc ---
[INFO] skip non existing resourceDirectory /Users/raulzunigagonzalez/Library/Mobile Documents/com~apple~CloudDoc
s/Raul/DUOC/DUOC - Desarrollo de Aplicaciónes /08 - Bimestre/02 - Bakend III/05 - Implementando el patrón arquit
ectonico Backend for Frontend/Exp2_S5_RaulZuniga/src/test/resources
[INFO]
[INFO] --- compiler:3.14.1:testCompile (default-testCompile) @ Duoc ---
[INFO] Nothing to compile - all classes are up to date.
[INFO]
[INFO] <<< spring-boot:3.5.7:run (default-cli) < test-compile @ Duoc <<<
[INFO]
[INFO]
[INFO] --- spring-boot:3.5.7:run (default-cli) @ Duoc ---
[INFO] Attaching agents: []

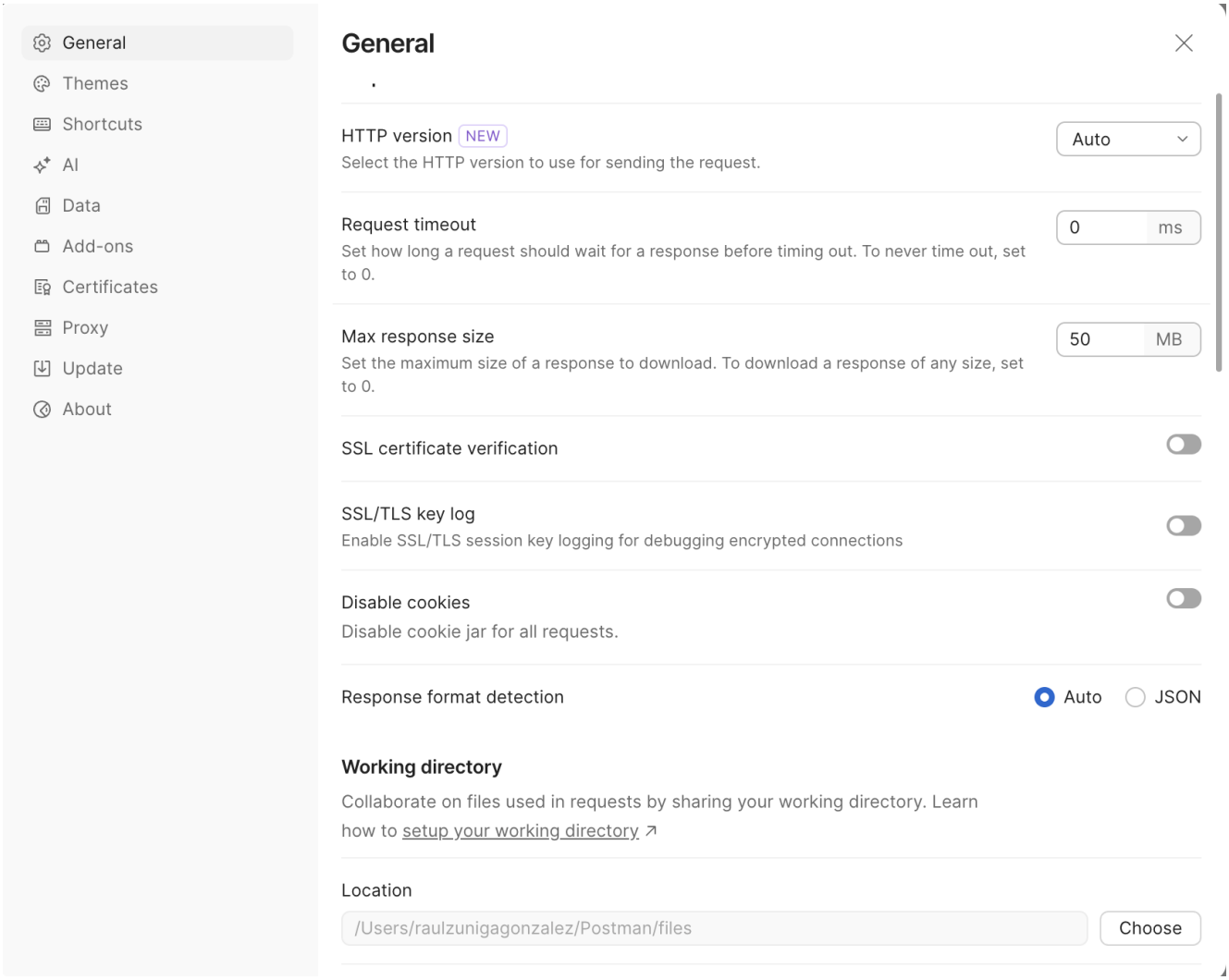
  .   ____          _            __ _ _
 /\\ / ___'_ __ _ _(_)_ __  __ _ \ \ \ \
( ( )\___ | '_ | '_| | '_ \/ _` | \ \ \ \
 \\/  ___)| |_)| | | | | || (_| |  ) ) ) )
  '  |____| .__|_| |_|_| |_\__, | / / / /
 =========|_|==============|___/=/_/_/_/

 :: Spring Boot ::                (v3.5.7)

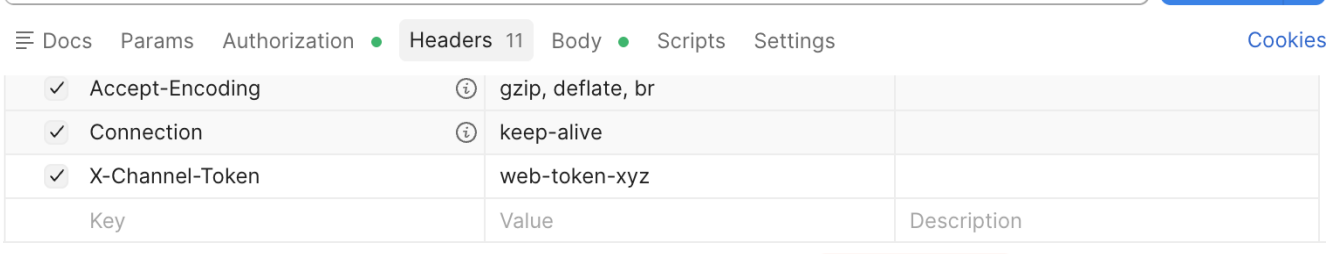
2026-09-14T21:22:51.112-03:00  INFO 71551 --- [Duoc] [           main] com.backen3.Duoc.DuocApplication
: Starting DuocApplication using Java 25 with PID 71551 (/Users/raulzunigagonzalez/Library/Mobile Documents/com~
apple~CloudDocs/Raul/DUOC/DUOC - Desarrollo de Aplicaciónes /08 - Bimestre/02 - Bakend III/05 - Implementando el
 patrón arquitectonico Backend for Frontend/Exp2_S5_RaulZuniga/target/classes started by raulzunigagonzalez in /
Users/raulzunigagonzalez/Library/Mobile Documents/com~apple~CloudDocs/Raul/DUOC/DUOC - Desarrollo de Aplicacióne
s /08 - Bimestre/02 - Bakend III/05 - Implementando el patrón arquitectonico Backend for Frontend/Exp2_S5_RaulZu
niga)
2026-09-14T21:22:51.113-03:00  INFO 71551 --- [Duoc] [           main] com.backen3.Duoc.DuocApplication
: No active profile set, falling back to 1 default profile: "default"
2026-09-14T21:22:51.376-03:00  INFO 71551 --- [Duoc] [           main] .s.d.r.c.RepositoryConfigurationDelegate
: Bootstrapping Spring Data JPA repositories in DEFAULT mode.
2026-09-14T21:22:51.387-03:00  INFO 71551 --- [Duoc] [           main] .s.d.r.c.RepositoryConfigurationDelegate
: Finished Spring Data repository scanning in 6 ms. Found 0 JPA repository interfaces.
2026-09-14T21:22:51.648-03:00  INFO 71551 --- [Duoc] [           main] o.s.b.w.embedded.tomcat.TomcatWebServer
: Tomcat initialized with port 8443 (https)
2026-09-14T21:22:51.656-03:00  INFO 71551 --- [Duoc] [           main] o.apache.catalina.core.StandardService
: Starting service [Tomcat]
2026-09-14T21:22:51.656-03:00  INFO 71551 --- [Duoc] [           main] o.apache.catalina.core.StandardEngine
: Starting Servlet engine: [Apache Tomcat/10.1.48]
2026-09-14T21:22:51.680-03:00  INFO 71551 --- [Duoc] [           main] o.a.c.c.C.[Tomcat].[localhost].[/]
: Initializing Spring embedded WebApplicationContext
```

```
2026-09-14T21:22:51.681-03:00  INFO 71551 --- [Duoc] [           main] w.s.c.ServletWebServerApplicationContext
: Root WebApplicationContext: initialization completed in 544 ms
2026-09-14T21:22:51.720-03:00  INFO 71551 --- [Duoc] [           main] com.zaxxer.hikari.HikariDataSource
: HikariPool-1 - Starting...
2026-09-14T21:22:51.824-03:00  INFO 71551 --- [Duoc] [           main] com.zaxxer.hikari.pool.HikariPool
: HikariPool-1 - Added connection com.mysql.cj.jdbc.ConnectionImpl@3d620a1
2026-09-14T21:22:51.824-03:00  INFO 71551 --- [Duoc] [           main] com.zaxxer.hikari.HikariDataSource
: HikariPool-1 - Start completed.
2026-09-14T21:22:51.907-03:00  INFO 71551 --- [Duoc] [           main] o.hibernate.jpa.internal.util.LogHelper
: HHH000204: Processing PersistenceUnitInfo [name: default]
2026-09-14T21:22:51.935-03:00  INFO 71551 --- [Duoc] [           main] org.hibernate.Version
: HHH000412: Hibernate ORM core version 6.6.33.Final
2026-09-14T21:22:51.951-03:00  INFO 71551 --- [Duoc] [           main] o.h.c.internal.RegionFactoryInitiator
: HHH000026: Second-level cache disabled
2026-09-14T21:22:52.078-03:00  INFO 71551 --- [Duoc] [           main] o.s.o.j.p.SpringPersistenceUnitInfo
: No LoadTimeWeaver setup: ignoring JPA class transformer
2026-09-14T21:22:52.120-03:00  WARN 71551 --- [Duoc] [           main] org.hibernate.orm.deprecation
: HHH90000025: MySQLDialect does not need to be specified explicitly using 'hibernate.dialect' (remove the prope
rty setting and it will be selected by default)
2026-09-14T21:22:52.125-03:00  INFO 71551 --- [Duoc] [           main] org.hibernate.orm.connections.pooling
: HHH10001005: Database info:
        Database JDBC URL [Connecting through datasource 'HikariDataSource (HikariPool-1)']
        Database driver: undefined/unknown
        Database version: 8.0.39
        Autocommit mode: undefined/unknown
        Isolation level: undefined/unknown
        Minimum pool size: undefined/unknown
        Maximum pool size: undefined/unknown
2026-09-14T21:22:52.252-03:00  INFO 71551 --- [Duoc] [           main] o.h.e.t.j.p.i.JtaPlatformInitiator
: HHH000489: No JTA platform available (set 'hibernate.transaction.jta.platform' to enable JTA platform integrat
ion)
2026-09-14T21:22:52.254-03:00  INFO 71551 --- [Duoc] [           main] j.LocalContainerEntityManagerFactoryBean
: Initialized JPA EntityManagerFactory for persistence unit 'default'
2026-09-14T21:22:52.327-03:00  WARN 71551 --- [Duoc] [           main] o.s.b.c.s.b.FaultTolerantStepBuilder
: Asynchronous TaskExecutor detected with ItemStream reader.  This is probably an error, and may lead to incorre
ct restart data being stored.
2026-09-14T21:22:52.337-03:00  WARN 71551 --- [Duoc] [           main] o.s.b.c.s.b.FaultTolerantStepBuilder
: Asynchronous TaskExecutor detected with ItemStream reader.  This is probably an error, and may lead to incorre
ct restart data being stored.
2026-09-14T21:22:52.341-03:00  WARN 71551 --- [Duoc] [           main] o.s.b.c.s.b.FaultTolerantStepBuilder
: Asynchronous TaskExecutor detected with ItemStream reader.  This is probably an error, and may lead to incorre
ct restart data being stored.
2026-09-14T21:22:52.360-03:00  WARN 71551 --- [Duoc] [           main] JpaBaseConfiguration$JpaWebConfiguration
: spring.jpa.open-in-view is enabled by default. Therefore, database queries may be performed during view render
ing. Explicitly configure spring.jpa.open-in-view to disable this warning
2026-09-14T21:22:52.586-03:00  INFO 71551 --- [Duoc] [           main] o.a.t.util.net.NioEndpoint.certificate
: Connector [https-jsse-nio-8443], TLS virtual host [_default_], certificate type [UNDEFINED] configured from ke
ystore [/Users/raulzunigagonzalez/.keystore] using alias [banco-xyz] with trust store [null]
2026-09-14T21:22:52.591-03:00  INFO 71551 --- [Duoc] [           main] o.s.b.w.embedded.tomcat.TomcatWebServer
: Tomcat started on port 8443 (https) with context path '/'
2026-09-14T21:22:52.595-03:00  INFO 71551 --- [Duoc] [           main] com.backen3.Duoc.DuocApplication
: Started DuocApplication in 1.647 seconds (process running for 1.778)
2026-09-14T21:22:52.597-03:00  INFO 71551 --- [Duoc] [           main] com.backen3.Duoc.batch.CsvBatchConfig
: Iniciando Job: reporteTransaccionesDiariasJob con fechaProceso=2024-01
2026-09-14T21:22:52.640-03:00  WARN 71551 --- [Duoc] [           main] com.backen3.Duoc.batch.CsvBatchConfig
: Job omitido porque ya fue completado con los mismos parametros: reporteTransaccionesDiariasJob | fechaProceso=
2024-01
2026-09-14T21:22:52.640-03:00  INFO 71551 --- [Duoc] [           main] com.backen3.Duoc.batch.CsvBatchConfig
: Iniciando Job: calculoInteresesMensualesJob con fechaProceso=2024-01
2026-09-14T21:22:52.646-03:00  WARN 71551 --- [Duoc] [           main] com.backen3.Duoc.batch.CsvBatchConfig
2026-09-14T21:22:52.646-03:00  WARN 71551 --- [Duoc] [           main] com.backen3.Duoc.batch.CsvBatchConfig
: Job omitido porque ya fue completado con los mismos parametros: calculoInteresesMensualesJob | fechaProceso=20
24-01
2026-09-14T21:22:52.646-03:00  INFO 71551 --- [Duoc] [           main] com.backen3.Duoc.batch.CsvBatchConfig
: Iniciando Job: estadosCuentaAnualesJob con fechaProceso=2024-01
2026-09-14T21:22:52.650-03:00  WARN 71551 --- [Duoc] [           main] com.backen3.Duoc.batch.CsvBatchConfig
: Job omitido porque ya fue completado con los mismos parametros: estadosCuentaAnualesJob | fechaProceso=2024-01
```

Ahora, si nos vamos a POSTMAN y antes de probar las APIs que hemos creados y validadas con nuestro certificado, debemos ingresar a los ajustes de POSTMAN y desactivar la validación SSL certifcate verification. Con esto vamos a poder probar los endpoints con POSTMAN



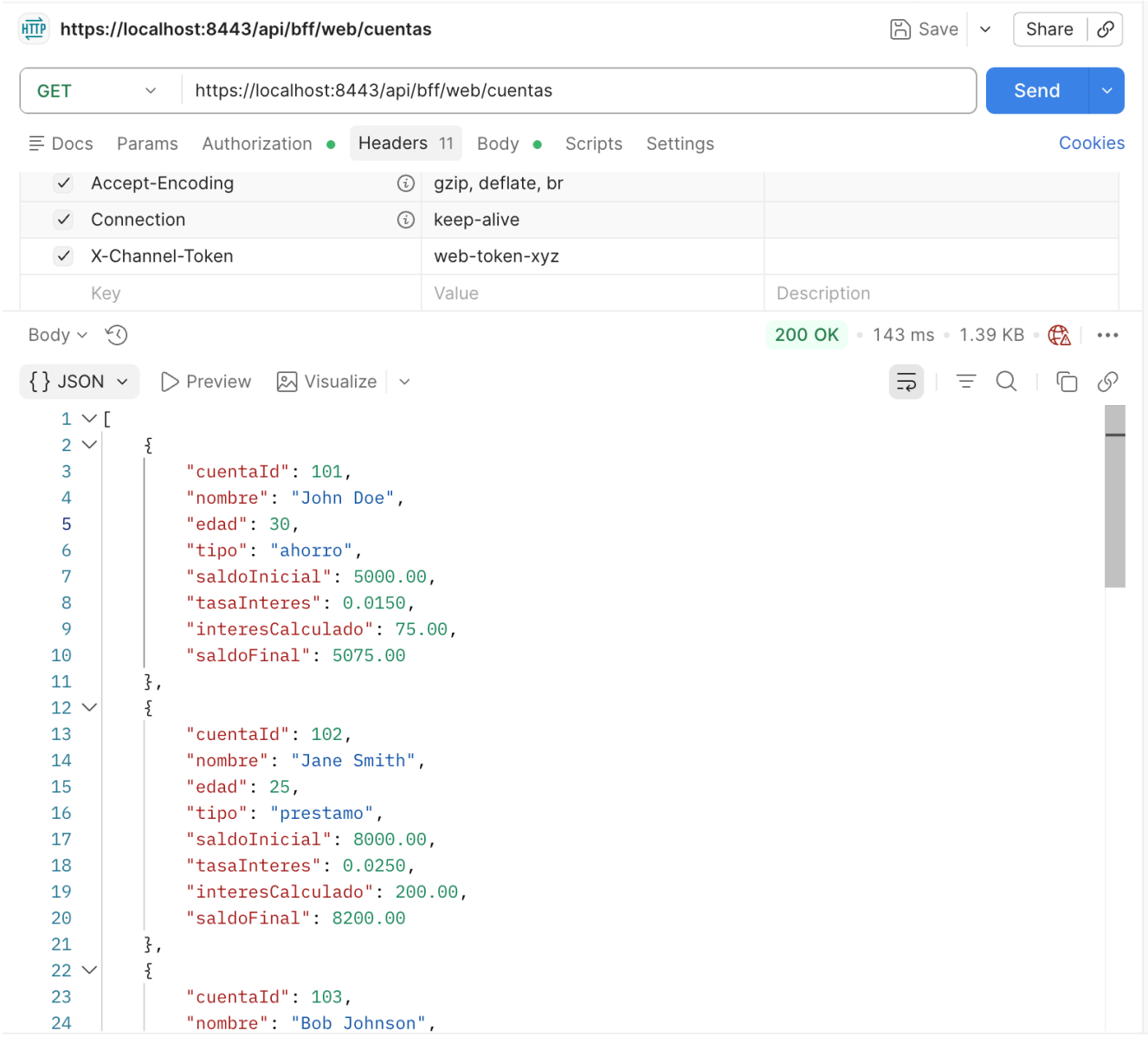
Ahora, vamos a ir donde dice Headers y donde dice Key, vamos a escribir X-Channel-Token y en Value escribiremos web-token-xyz, esto nos permitirá visualizar el BFF web



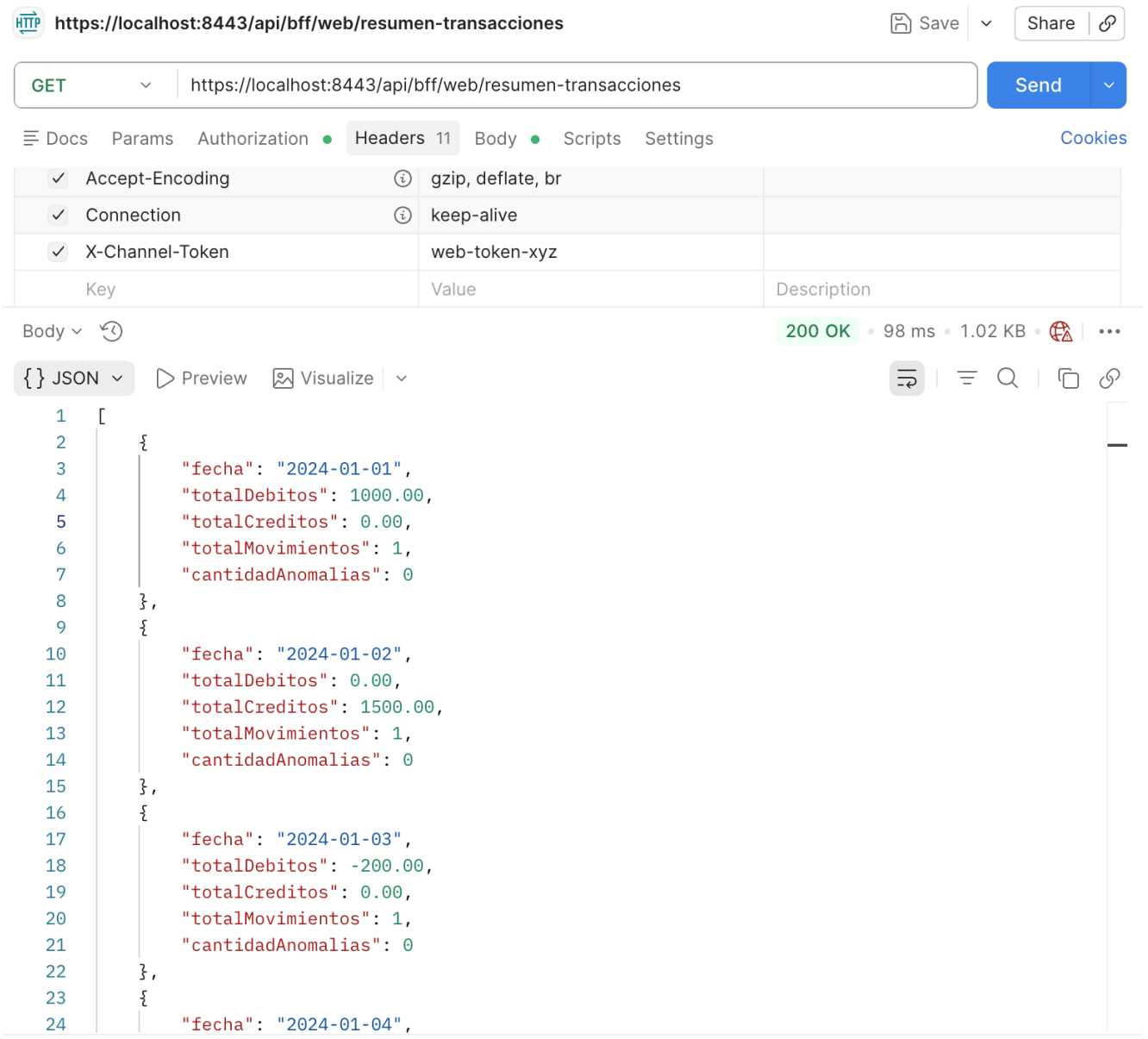
Ahora lo que vamos a hacer es probar los distintos BFF

1- BFF Web

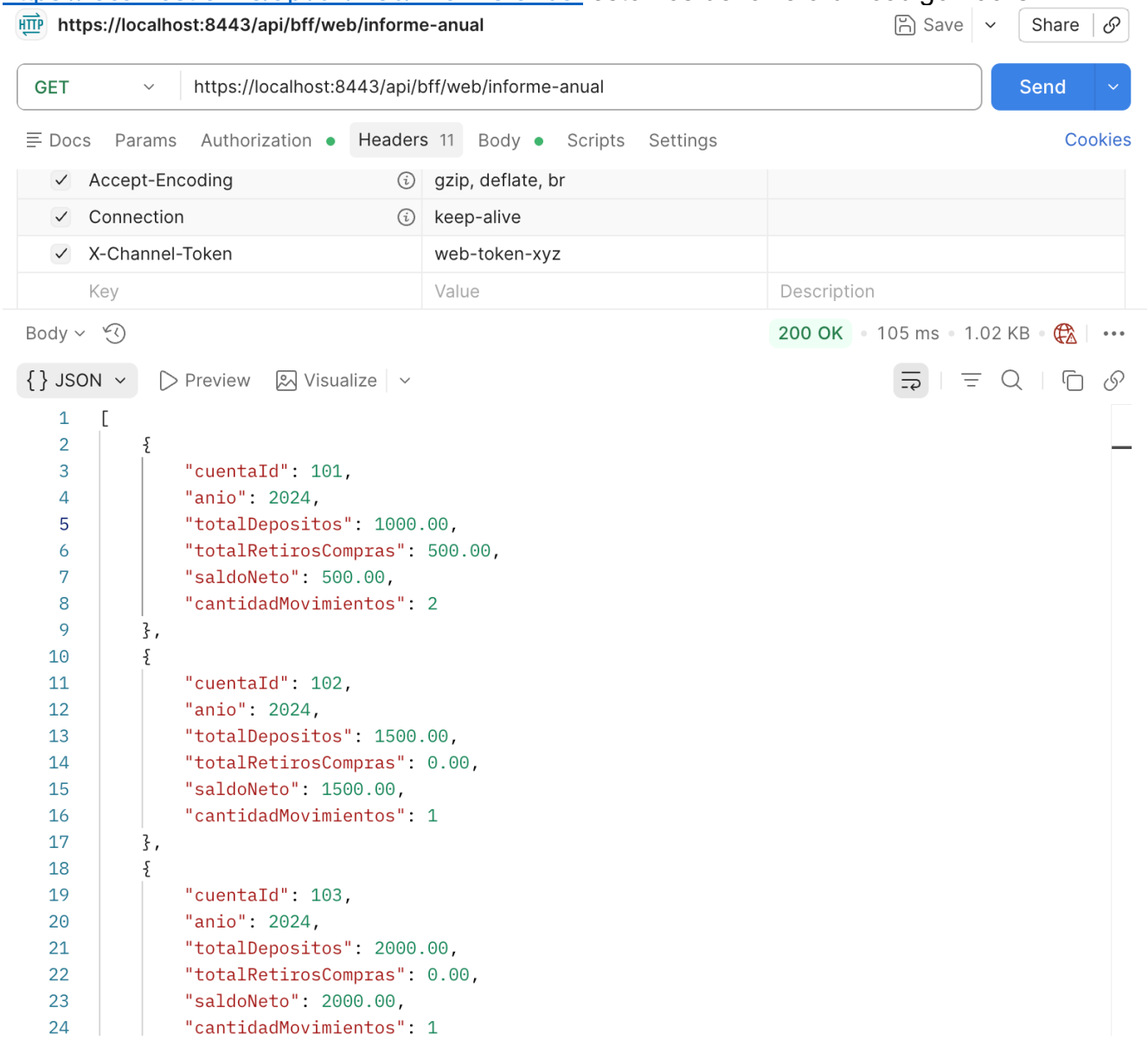
Para ello vamos a seleccionar el método GET y vamos a escribir la siguiente ruta https://localhost:8443/api/bff/web/cuentas y le vamos a dar al botón SEND. Esto nos arrojará como resultado un código 200 OK.



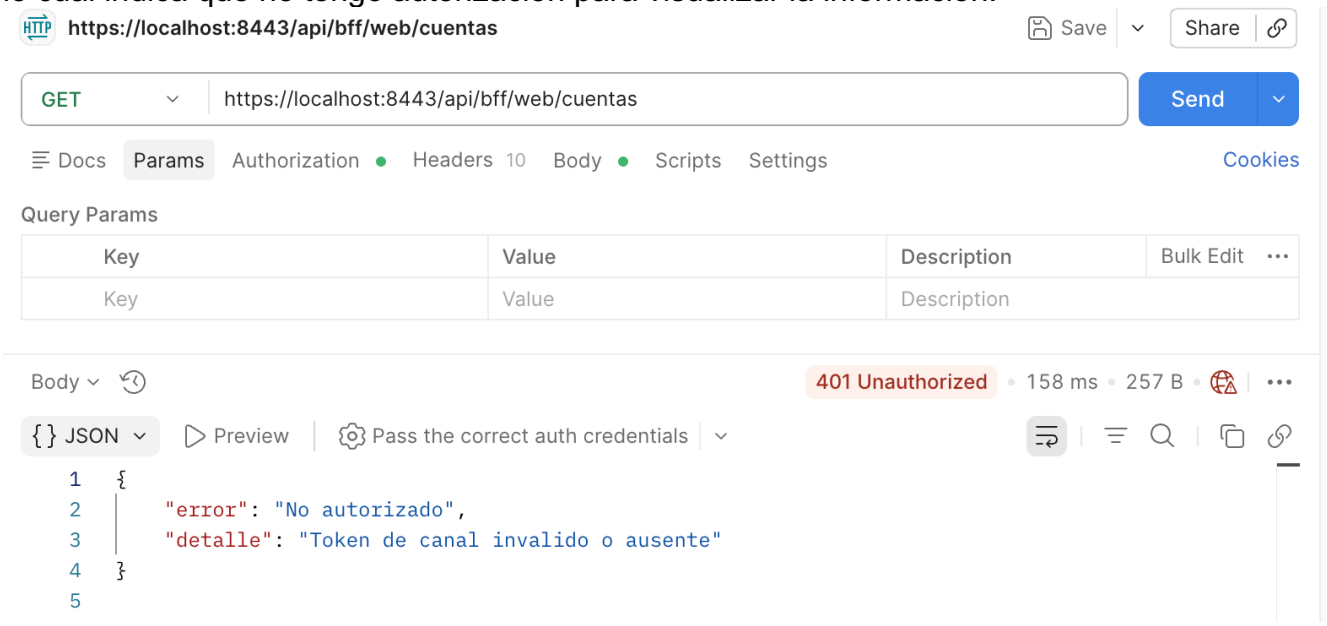
También probamos los endpoints para visualizar las apis de los resumen de transacciones, para lo cual mantenemos el método GET y escribimos la siguiente dirección URL https://localhost:8443/api/bff/web/resumen-transacciones esto nos devolverá un código 200OK



Ahora vamos a probar la api que entrega la información web del informe anual, para ello mantenemos el método GET y seleccionamos la siguiente URL https://localhost:8443/api/bff/web/informe-anual esto nos devolverá un código 200 OK



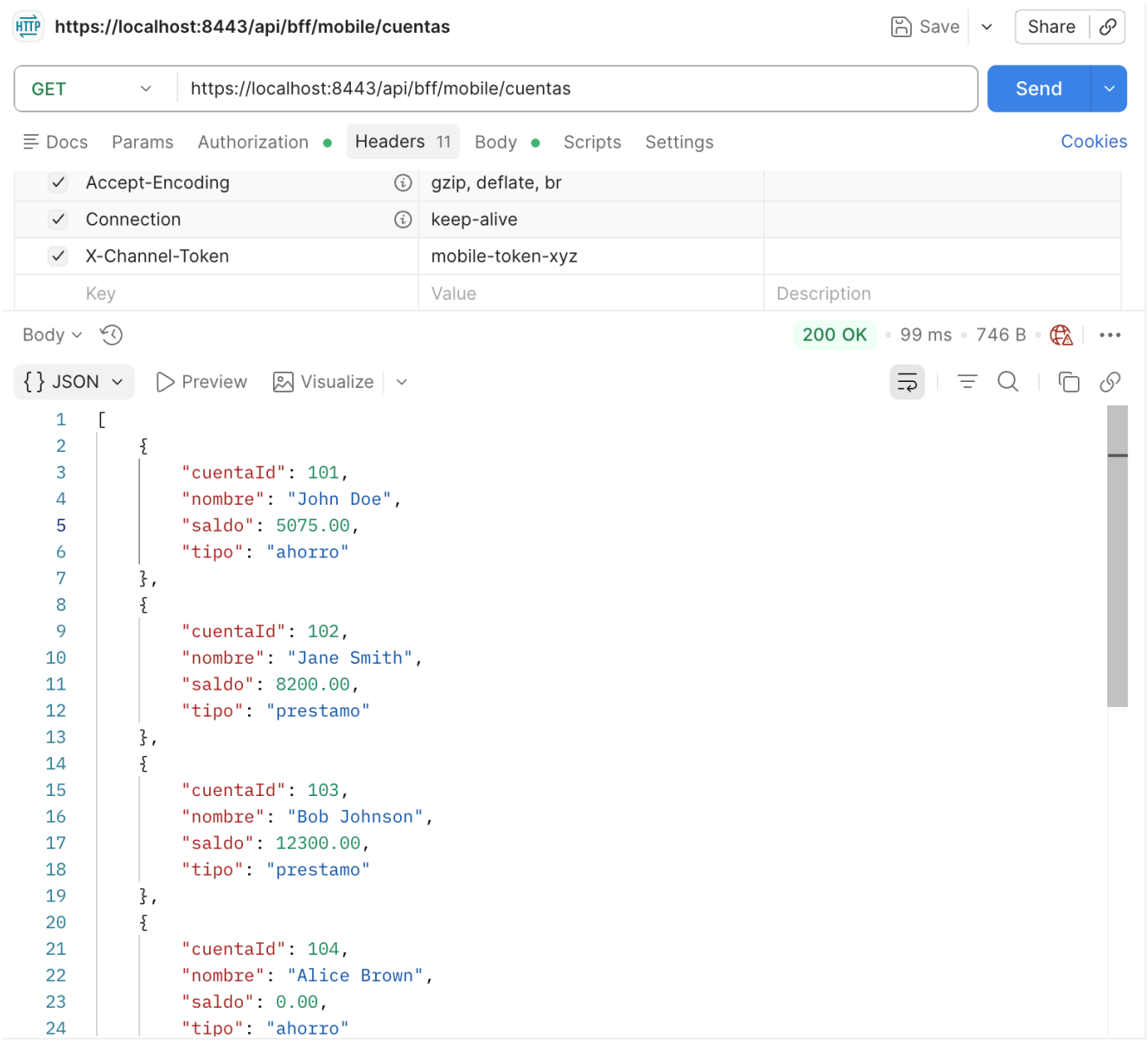
Si eliminamos el token, para ingresar, podremos ver que nos devolverá un 401 Unauthorized, lo cual indica que no tengo autorización para visualizar la información.



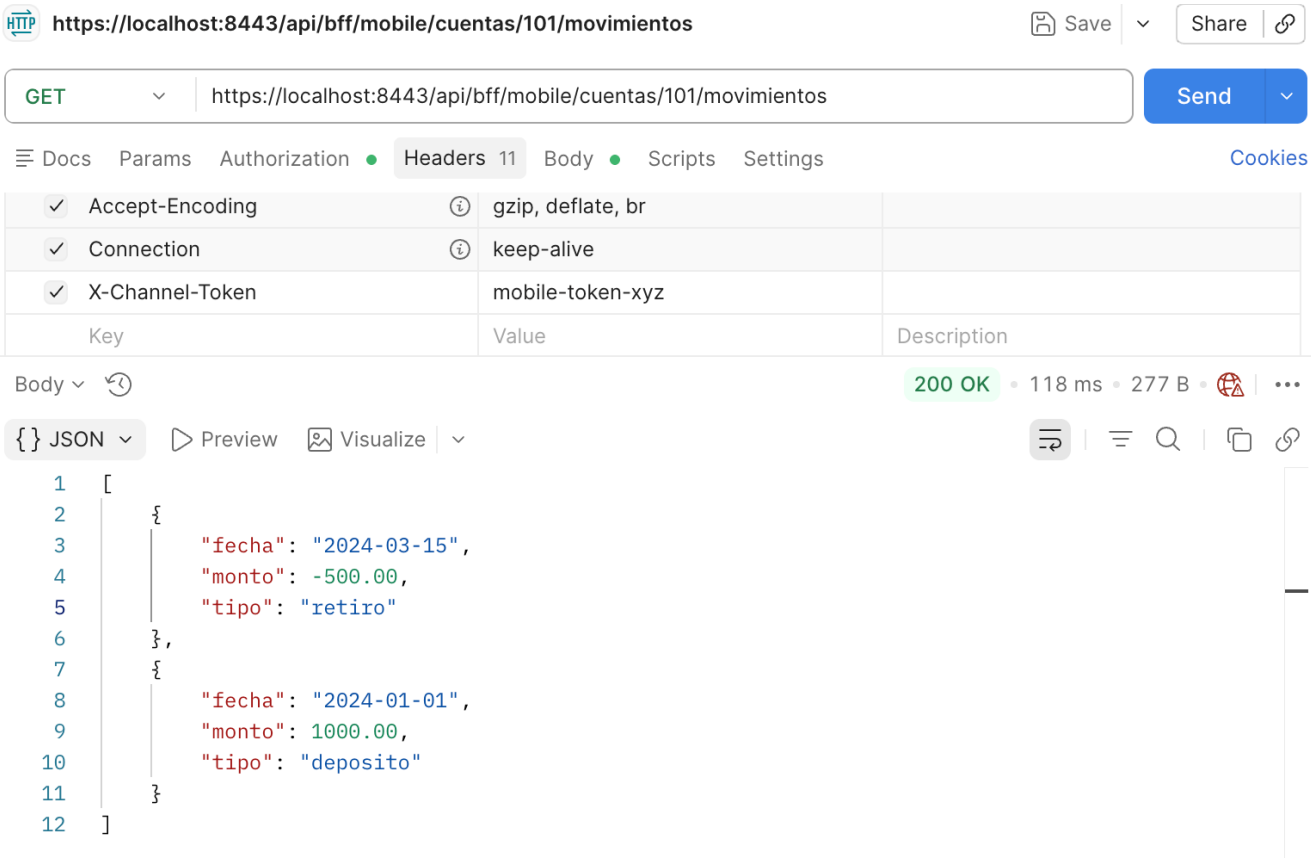
2- BFF Móvil

Ahora, vamos a trabajar con el BFF de Móvil, para ello debemos de modificar el token que se realizó para el BFF Web, para ello vamos a ir donde dice Headers y donde dice Key, vamos a escribir X-Channel-Token y en Value escribiremos mobile-token-xyz, esto nos permitirá visualizar el BFF Móvil

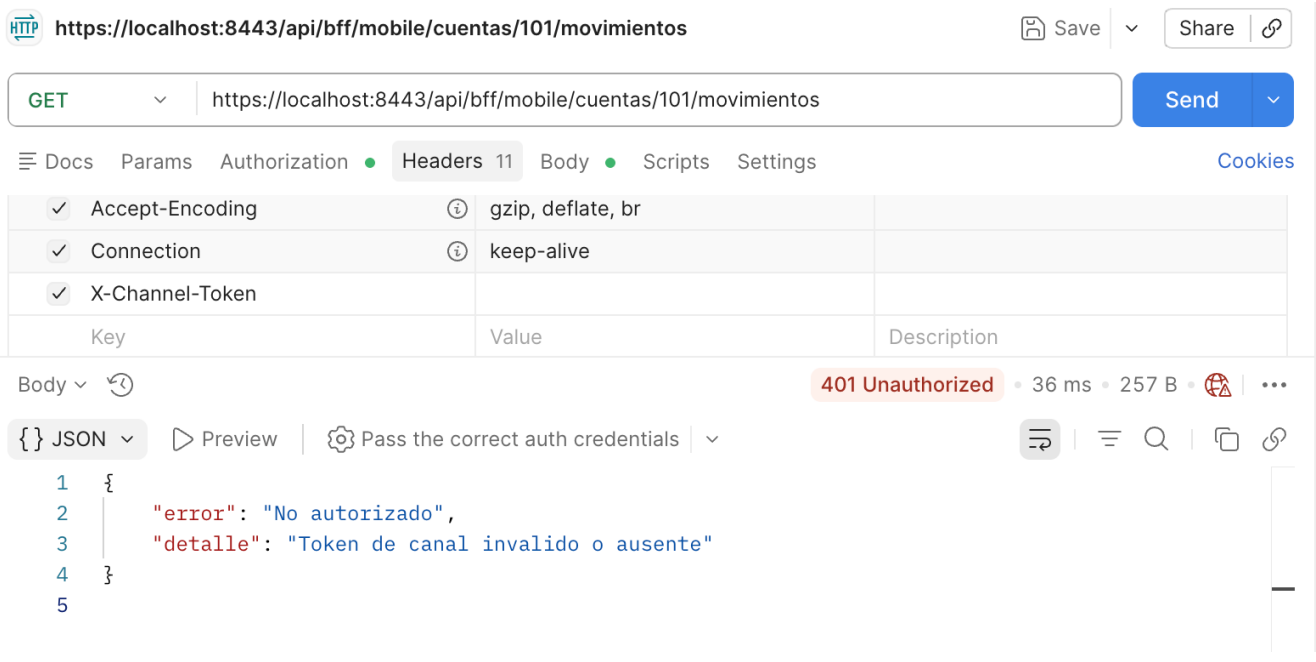
Ahora vamos a seleccionar el método GET y seleccionaremos la URL https://localhost:8443/api/bff/mobile/cuentas, el cual nos devolverá un código 200 OK



Ahora si quisiéramos visualizar los movimientos del BFF Móvil, debemos mantener el método GET y seleccionar la siguiente URL https://localhost:8443/api/bff/mobile/cuentas/101/movimientos, al darle al botón Send, nos devolverá el código 200 OK



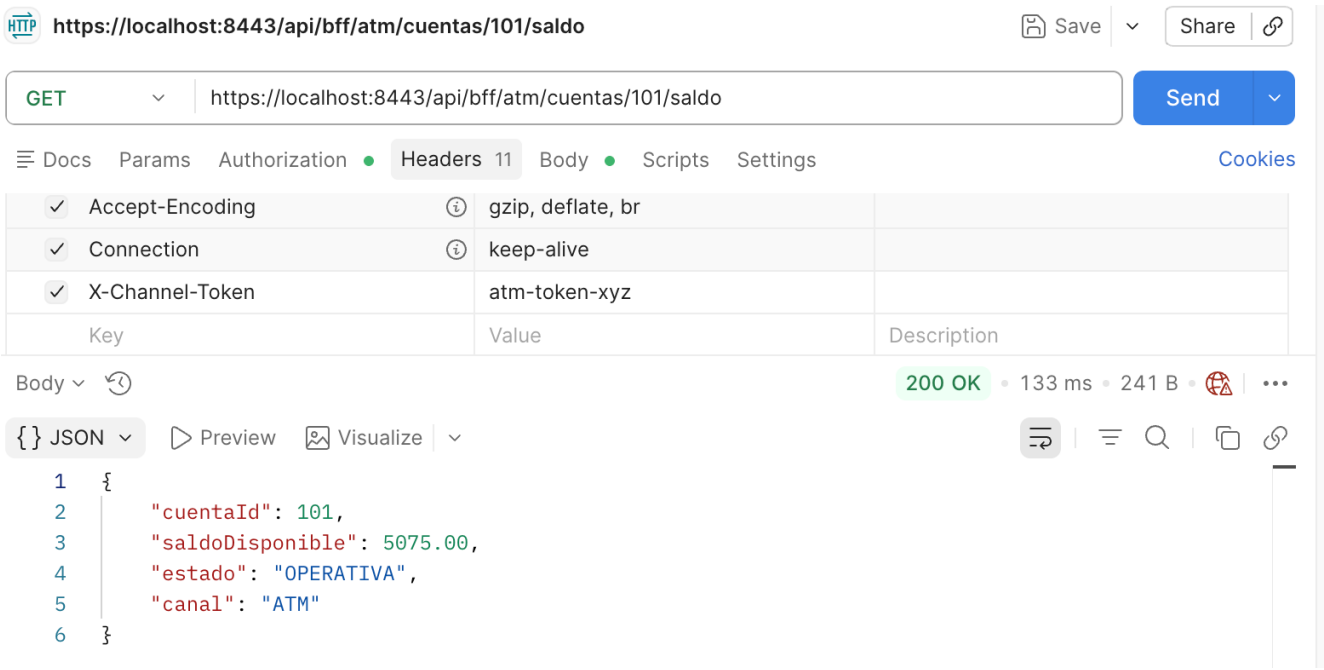
Al igual que en el caso web, si no tiene el token correcto, no nos permitirá visualizar la información, devolviéndonos un código 401 Unauthorized.



3- BFF Cajero

Ahora, vamos a trabajar con el BFF del Cajero, para ello debemos de modificar el token que se realizó para el BFF Cajero, para ello vamos a ir donde dice Headers y donde dice Key, vamos a escribir X-Channel-Token y en Value escribiremos atm-token-xyz, esto nos permitirá visualizar el BFF Cajero.

Ahora en POSTMAN vamos a seleccionar el método GET y vamos a escribir la siguiente URL GET https://localhost:8443/api/bff/atm/cuentas/101/saldo si le damos al botón send, nos devolverá un código 200 OK donde visualizaremos el saldo que tiene el usuario



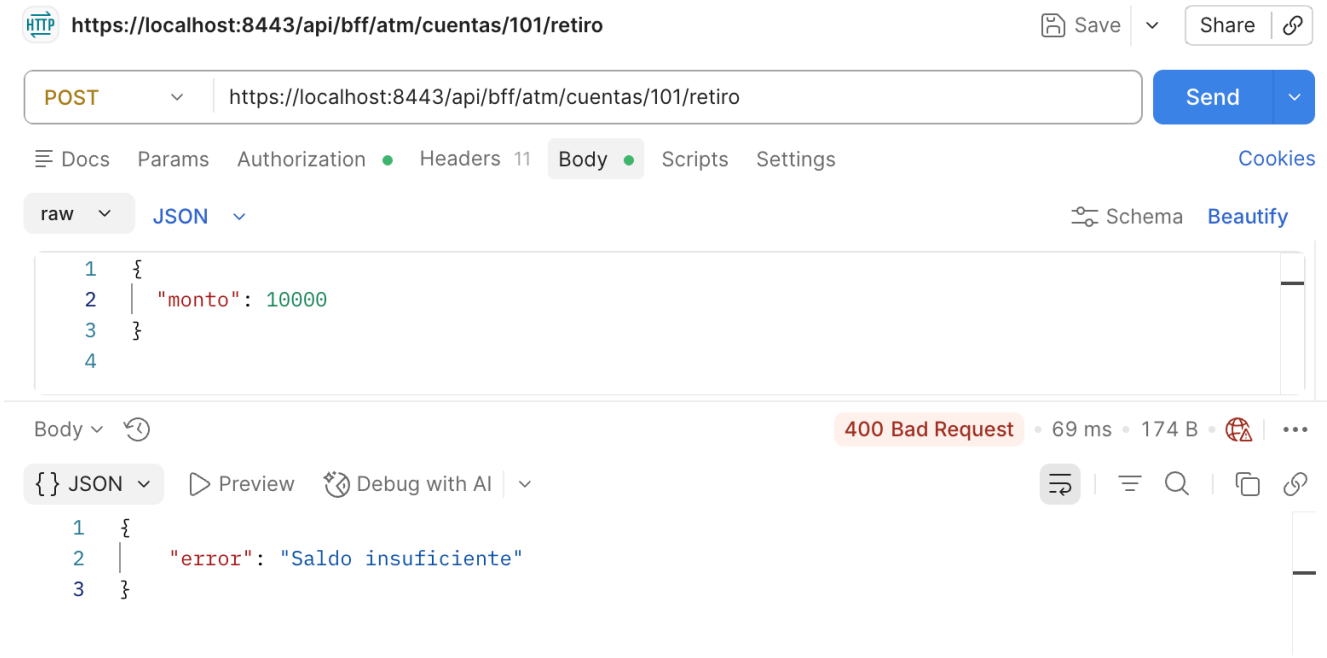
Como es un cajero, vamos a simular como se vería al hacer un retiro, para ello ahora nos vamos donde dice Body y escribiremos el siguiente JSON:

```
{
  "monto": 1000
}
```

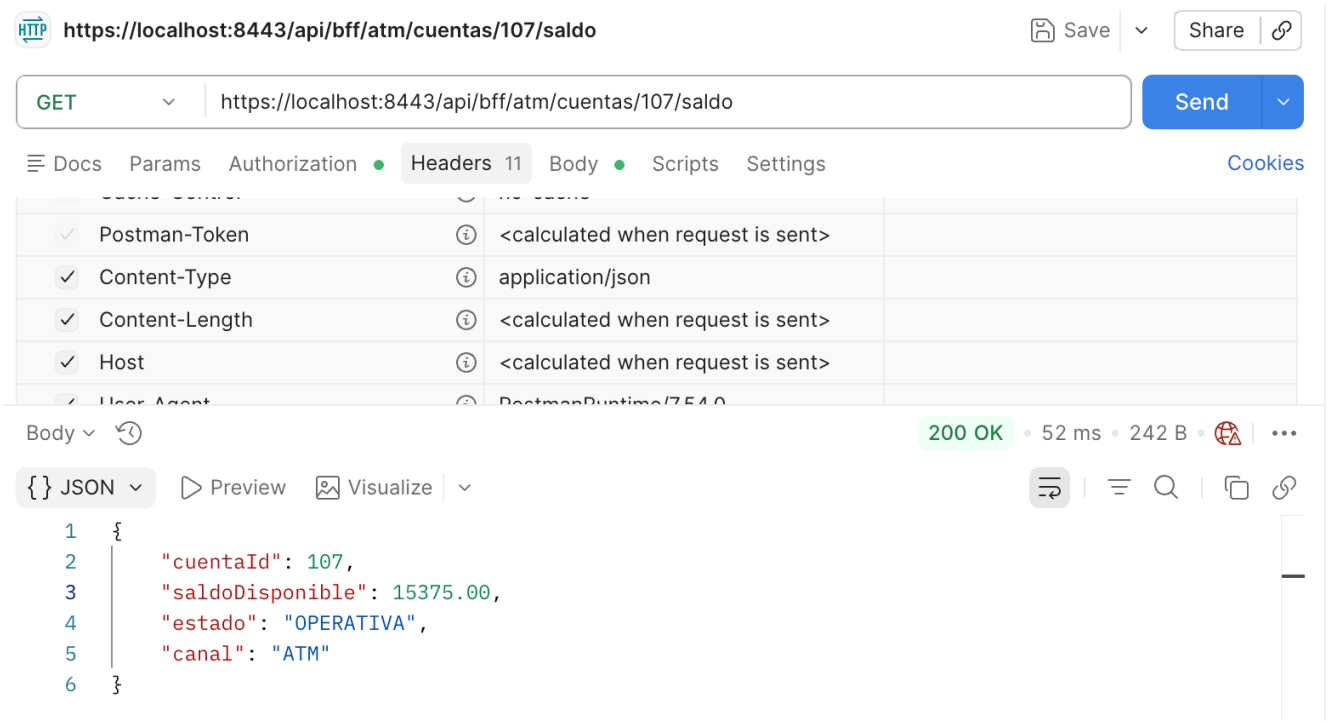
Ahora modificaremos el método GET por POST, también, cambiaremos el URL que trabajamos anteriormente y modificaremos donde dice saldo a retiro, para finalizar, le daremos clic al botón Send, esto nos debería devolver un código 400 Bad Request con un mensaje de error que nos indica que el cajero solo permite retiros de $10.000 algo muy común con algunos cajeros



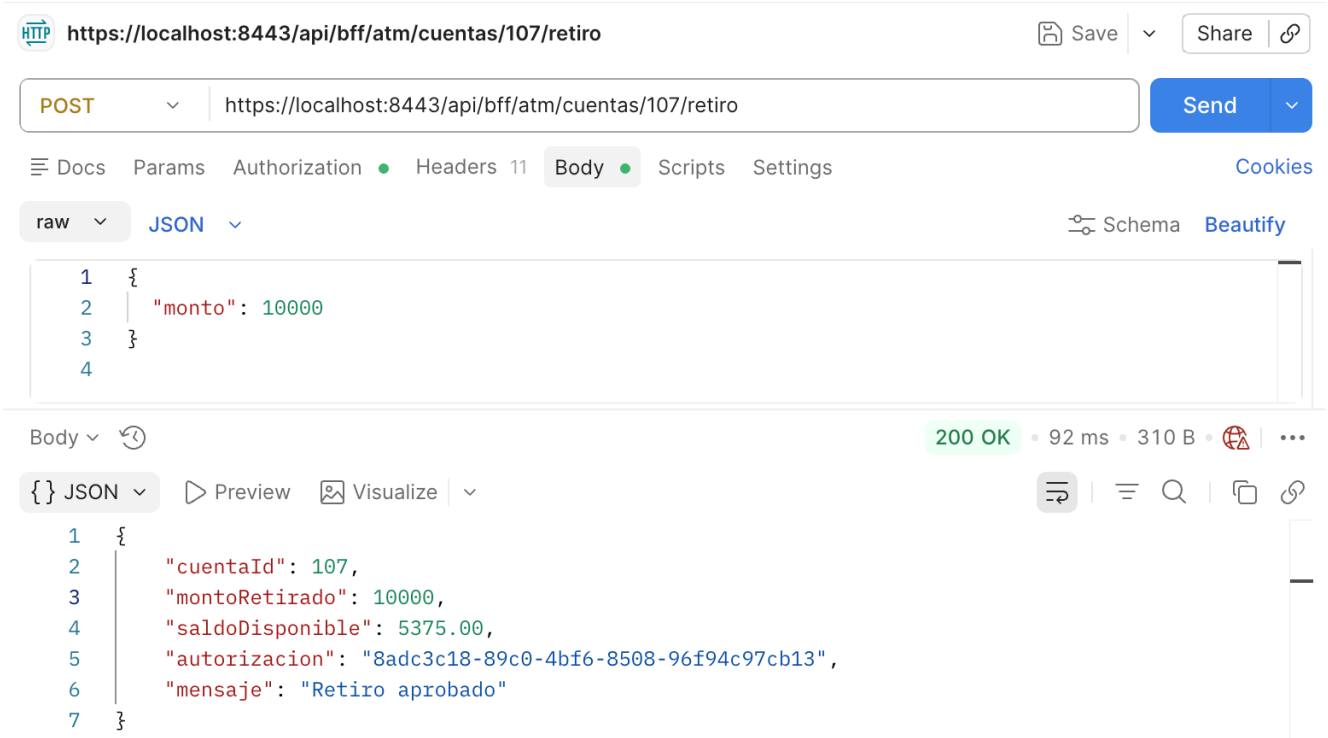
Si modificamos JSON a $10.000 ahora nos va a devolver un 400 Bad Request debido a que el usuario tiene un saldo insuficiente para poder retirar $10.000.



Para poder finalizar, modificamos el usuario 101 al 107 para realizar el retiro, y esto es debido a que dicho usuario tiene saldo suficiente como para hacer el retiro de $10.000



Ahora volvemos a ejecutar el retiro pero ahora con el usuario 107, esto nos devolverá un código 200 Ok, nos indicará el monto retirado y nos mostrará a su vez el valor del saldo disponible actualmente con un código de autorización y mensaje, que indica que el retiro ha sido aprobado.




Duoc UC
ONLINE